

OFPPT

Office de la Formation Professionnelle et de la Promotion du Travail

Cité des Métiers et des Compétences — Béni Mellal

RAPPORT DE STAGE

CMC Data API

Conception et développement d'une API REST de centralisation, de filtrage avancé et d'import de données pour la gestion académique du centre

Filière : Développement Digital — Option Développement Web Full Stack

Niveau : Technicien Spécialisé (TS)

Réalisé par :

EL-BIYANI Achraf

OUMOUS Mouad

Encadré par : **LACHHAB FAISSAL**

Tuteur de stage (entreprise) : CMC - BMK

Année de formation 2025 / 2026

Remerciements

Au terme de ce stage, je tiens à exprimer ma sincère gratitude envers toutes les personnes qui ont contribué, de près ou de loin, à la réussite de cette expérience professionnelle et à l'élaboration de ce rapport.

Je remercie chaleureusement l'ensemble de l'équipe pédagogique et administrative de la Cité des Métiers et des Compétences (CMC) de Béni Mellal, et en particulier le Pôle Digital et Intelligence Artificielle, pour m'avoir accueilli dans un cadre de travail stimulant et pour la confiance qu'elle m'a accordée tout au long de ce projet.

Mes remerciements s'adressent également à mon tuteur de stage pour son encadrement, sa disponibilité, ses conseils techniques avisés et le partage de son expérience, qui m'ont permis de mener ce projet dans de bonnes conditions et de monter en compétence sur des sujets concrets d'ingénierie logicielle.

Je n'oublie pas non plus mes formateurs au sein du centre, dont l'enseignement m'a fourni les bases théoriques et pratiques indispensables à la réalisation de ce travail, ainsi que l'ensemble du personnel de la coordination pédagogique et de la scolarité qui a accepté de partager avec moi les données et les contraintes réelles de leur travail quotidien, contraintes qui sont à l'origine même de ce projet.

Enfin, je remercie ma famille et mes camarades de promotion pour leur soutien constant durant cette période de stage.

Résumé

La Cité des Métiers et des Compétences (CMC) de Béni Mellal gère, comme l'ensemble du réseau de l'OFPPT, un volume important de données académiques : pôles de formation, filières, années, groupes, modules, formateurs, stagiaires, séances de cours, évaluations, etc. Ces données sont aujourd'hui dispersées dans plusieurs classeurs Excel produits par différents services (scolarité, ressources humaines, coordination pédagogique), sans schéma commun, sans contrôle de cohérence et sans possibilité de filtrage, de croisement ou de recherche avancée. Cette situation rend difficile toute exploitation transversale de l'information et toute automatisation des traitements (suivi de l'avancement pédagogique, statistiques, tableaux de bord, etc.).

Ce stage avait pour objectif de concevoir et de développer une API REST, baptisée CMC Data API, destinée à devenir la source de vérité unique des données du centre. Le projet repose sur le framework Laravel (PHP 8.3) et expose, sous une architecture en couches (**Routes** → **Form Requests** → **Filtres** → **Services** → **Resources**), un ensemble d'endpoints versionnés permettant de consulter, filtrer, trier, paginer et faire évoluer ces données de façon fiable et sécurisée. Un module d'import dédié permet par ailleurs de charger automatiquement les fichiers **Excel/CSV** existants (listes de formateurs, listes de stagiaires) en reconstruisant la hiérarchie académique sous-jacente et en détectant les anomalies ligne par ligne.

Le présent rapport décrit l'ensemble de la démarche suivie : analyse du contexte et de la problématique, rédaction du cahier des charges, conception du modèle de données et de l'architecture logicielle, puis description détaillée du contrat d'API (routes, formats de requêtes, règles de validation, formats de réponses, mécanismes de filtrage avancé), avant de présenter les aspects transverses de sécurité, de performance (mise en cache) et de tests automatisés mis en œuvre.

Table des matières

Remerciements	2
Résumé	3
Chapitre 1 — Présentation du contexte et problématique	6
1.1 Présentation de l'organisme d'accueil	6
1.2 Contexte et constat de départ	6
1.3 Problématique	7
1.4 Objectifs du stage	7
Chapitre 2 — Cahier des charges	9
2.1 Cahier des charges fonctionnel (CDC-F)	9
2.1.1 Objectif	9
2.1.2 Périmètre fonctionnel — ressources métier	9
2.1.3 Fonctionnalités attendues	10
2.1.4 Acteurs	10
2.2 Cahier des charges technique (CDC-T)	10
2.2.1 Stack technique et contraintes	10
2.2.2 Architecture applicative imposée	11
2.2.3 Contrat d'API (résumé)	11
2.2.4 Modèle de données — résumé de la hiérarchie	11
2.2.5 Exigences non fonctionnelles	11
2.2.6 Critères d'acceptation	12
Chapitre 3 — Analyse et conception	13
3.1 Modèle de données	13
3.1.1 Contraintes d'intégrité notables	14
3.2 Architecture logicielle	14
3.2.1 Justification des choix d'architecture	15
3.3 Choix technologiques	15
Chapitre 4 — Routes, requêtes et réponses de l'API	17
4.1 Structure générale des routes	17
4.1.1 Ressources en lecture seule (index + show)	17
4.1.2 Ressources en CRUD complet	17
4.1.3 Routes d'agrégation hiérarchique (« cascade »)	18
4.1.4 Route d'import	19
4.2 Format des requêtes	19
4.2.1 Exemple — création d'une séance (StoreSeanceRequest)	19
4.2.2 Format des requêtes de mise à jour (sémantique PATCH)	20
4.2.3 Validation des paramètres de requête (filtres, tri, pagination)	20
4.2.4 Règles de validation personnalisées (defense in depth)	21
4.3 Format des réponses	21
4.3.1 Réponse d'une collection paginée	21
4.3.2 Relations chargées à la demande (?include=)	22
4.3.3 Codes de statut HTTP utilisés	22
4.4 Système de filtrage avancé	23
4.4.1 Principe de fonctionnement de QueryFilter	23
4.4.2 Exemple détaillé — filtrage des stagiaires	23
4.4.3 Filtres relationnels remontant la hiérarchie	24
4.4.4 Tableau récapitulatif des filtres disponibles par ressource	25

4.5 Gestion des erreurs	25
Chapitre 5 — Réalisation technique	27
5.1 Module d'import Excel / CSV	27
5.1.1 Format de la requête d'import	27
5.1.2 Normalisation des en-têtes de colonnes	27
5.1.3 Reconstruction automatique de la hiérarchie académique	27
5.1.4 Tolérance aux erreurs ligne par ligne	28
5.2 Sécurité	28
5.2.1 Identifiant de corrélation (Request ID)	29
5.2.2 Protection contre les écritures intersites	29
5.2.3 Limitation de débit (rate limiting)	29
5.2.4 Journal d'audit	29
5.2.5 Validation stricte de toutes les entrées	30
5.3 Performance et mise en cache	30
5.4 Interface web d'administration	30
Chapitre 6 — Tests et validation	31
6.1 Tests fonctionnels (Feature)	31
6.2 Données de test réalistes (factories et seeders)	31
6.3 Exemple de scénario de test	31
Chapitre 7 — Difficultés rencontrées et solutions apportées	33
Chapitre 8 — Conclusion et perspectives	34
8.1 Bilan du stage	34
8.2 Perspectives d'évolution (phase 2)	34

Chapitre 1 — Présentation du contexte et problématique

1.1 Présentation de l'organisme d'accueil

La Cité des Métiers et des Compétences (CMC) de Béni Mellal fait partie du réseau des nouveaux établissements de formation professionnelle déployés par l'OFPPT dans le cadre de la réforme de la formation professionnelle au Maroc. Le centre est structuré en plusieurs « pôles » de compétences (Pôle Digital et Intelligence Artificielle, Pôle Gestion et Commerce, Pôle Bâtiment et Travaux Publics, Pôle Mécanique et Électromécanique, etc.), chacun regroupant plusieurs filières de formation diplômante de niveau Technicien Spécialisé (TS).

Le stage s'est déroulé au sein du Pôle Digital et Intelligence Artificielle, qui héberge notamment les filières « Développement Digital » et « Infrastructure Digitale ». Ce pôle gère, pour chaque année de formation, plusieurs groupes de stagiaires, un ensemble de modules dispensés par des formateurs internes (statut OFPPT) ou vacataires, ainsi que la planification des séances de cours, de contrôle continu (CC) et d'épreuves de fin de module (EFM).

1.2 Contexte et constat de départ

Avant ce projet, la gestion des données du centre reposait presque exclusivement sur des classeurs Excel produits et mis à jour manuellement par différents intervenants : la base des formateurs (Base_Formateurs.xlsx), la base des stagiaires et des notes (BasePlateEvaluation*.xlsx, lister_minimized.xlsx) et le suivi de l'avancement pédagogique (AvancementProgramme*.xlsx). Ces fichiers ont permis l'analyse et la modélisation du domaine métier qui a servi de base à la conception de l'API, mais ils présentent en pratique plusieurs limites récurrentes, constatées sur le terrain :

- Absence de schéma commun : chaque fichier possède ses propres intitulés de colonnes (avec variantes orthographiques, accents, abréviations), ce qui complique tout rapprochement automatique entre fichiers.
- Duplication et incohérence des données : un même pôle, une même filière ou un même formateur peut apparaître sous plusieurs libellés légèrement différents selon le fichier ou la personne qui l'a saisi.
- Absence de contrôle d'intégrité : rien n'empêche, par exemple, qu'une séance soit rattachée à un module inexistant, qu'un stagiaire soit affecté à un groupe inconnu, ou qu'une note soit saisie pour une séance de type « cours » qui n'est pourtant pas évaluable.
- Impossibilité de filtrer, trier ou croiser les données de façon fiable : Excel permet des filtres manuels limités, mais aucune recherche structurée (par exemple « tous les stagiaires actifs d'un pôle donné, nés après une date donnée, dont la moyenne est inférieure à 10 ») n'est réalisable sans manipulation fastidieuse et sujette à erreur.
- Aucune API ni aucun point d'accès programmatique : les autres outils ou tableaux de bord du centre ne peuvent pas consommer ces données de façon automatisée, ce qui

empêche toute intégration avec d'éventuels services tiers (sites web, applications de suivi, outils de reporting).

Ce constat a fait émerger le besoin d'un système centralisé, fiable, interrogeable et capable d'absorber les données existantes sans imposer une re-saisie complète : c'est l'origine du projet CMC Data API.

1.3 Problématique

La problématique du stage peut être formulée ainsi : comment transformer un ensemble de fichiers Excel hétérogènes, dispersés et faiblement structurés, en une source de données unique, cohérente, filtrable et exploitable par d'autres services, tout en conservant la possibilité d'importer facilement les données existantes produites par les outils actuels du centre ?

Cette problématique se décline en plusieurs sous-questions techniques traitées dans ce rapport :

- Quel modèle de données relationnel permet de représenter fidèlement la hiérarchie académique du centre (Pôle → Filière → Année → Groupe/Module → Stagiaire, Affectation, Séance, Note) tout en respectant les contraintes d'intégrité réelles observées dans les données ?
- Quelle architecture logicielle permet d'exposer ces données de façon cohérente à la fois via une API REST (pour des services tiers) et via une interface web d'administration (pour les utilisateurs internes), sans dupliquer la logique métier ?
- Quel contrat d'API (routes, formats de requêtes, règles de validation, formats de réponses) garantit à la fois la souplesse de filtrage demandée et la robustesse face à des entrées malformées ou malveillantes ?
- Comment importer de façon fiable les fichiers Excel/CSV existants, malgré leurs incohérences de nommage, sans bloquer l'ensemble de l'import en cas d'erreur sur une seule ligne ?

1.4 Objectifs du stage

Les objectifs assignés à ce stage, tels que définis avec l'encadrement, étaient les suivants :

1. Analyser les fichiers Excel existants afin d'en extraire un modèle de données cohérent représentant l'ensemble du domaine académique du centre.
2. Rédiger un cahier des charges fonctionnel et technique formalisant le périmètre exact de l'API (ressources exposées, règles de gestion, exigences non fonctionnelles).
3. Concevoir et implémenter une API REST versionnée (Laravel 13 / PHP 8.3) exposant l'ensemble des ressources identifiées, avec pagination, tri, recherche et filtrage avancé contrôlé par liste blanche (« allowlist »).

4. Développer un module d'import Excel/CSV capable de charger les données existantes (formateurs, stagiaires) en reconstruisant automatiquement la hiérarchie académique manquante.
5. Mettre en place les mécanismes transverses indispensables à un service de données central : sécurité (limitation de débit, protection CSRF/CORS, journal d'audit), performance (cache des réponses), et tests automatisés.
6. Documenter l'ensemble du contrat d'API afin qu'il puisse être consommé par d'autres équipes ou applications du centre.

Chapitre 2 — Cahier des charges

Le cahier des charges ci-dessous formalise, à l'issue de la phase d'analyse, le périmètre fonctionnel et les contraintes techniques du projet CMC Data API. Il sert de référence tout au long du développement et de support pour la recette finale.

2.1 Cahier des charges fonctionnel (CDC-F)

2.1.1 Objectif

Le projet CMC Data API est une API REST exposant les données d'un centre de formation : structure académique, planification pédagogique et évaluations. Dans cette première phase, le système est volontairement ouvert (aucune authentification ni gestion de rôles n'est implémentée), l'objectif étant de valider d'abord le contrat de données et l'architecture ; la sécurisation par authentification est prévue en phase 2 (voir Chapitre 8 — Perspectives).

2.1.2 Périmètre fonctionnel — ressources métier

Le système couvre l'ensemble des entités suivantes, présentées avec leur rôle dans le domaine :

- **Pôle** (*Pole*) : département de formation du centre (ex. Pôle Digital et Intelligence Artificielle).
- **Espace** (*Espace*) : salle, laboratoire ou amphithéâtre rattaché à un pôle, avec une capacité d'accueil.
- **Niveau** (*Niveau*) : niveau de qualification OFPPT (TS, T, Q, FQ...).
- **Type de formation** (*TypeFormation*) : nature de la formation (Diplômante, Qualifiante...).
- **Filière** (*Filiere*) : programme de formation général, rattaché à un pôle, un niveau et un type de formation (ex. « Développement Digital »).
- **Année** (*Annee*) : année de formation au sein d'une filière, dont le libellé porte également l'éventuelle option de spécialisation (ex. « 2ème année — Option Développement Web Full Stack »).
- **Groupe** (*Groupe*) : groupe de stagiaires rattaché à une année de formation.
- **Module** (*Module*) : unité d'enseignement rattachée à une année, avec ses volumes horaires planifiés (présentiel, synchrone, asynchrone, total).
- **Affectation** (*Affectation*) : pivot central reliant un Groupe, un Module et un (ou deux) Formateur(s), avec le mode pédagogique et les heures affectées.
- **Séance** (*Seance*) : session de cours planifiée pour une affectation, typée (cours, contrôle continu, épreuve de fin de module, examen) et positionnée dans le temps via un créneau horaire et, le cas échéant, un espace physique.
- **Créneau horaire** (*TimeRange*) : plage horaire standard utilisée pour planifier les séances.

- **Formateur** (*Formateur*) : enseignant rattaché à un pôle, avec son statut (OFPPT, Vacataire, Contractuel) et son éventuelle mutualisation vers un autre pôle.
- **Stagiaire** (*Stagiaire*) : apprenant rattaché à un groupe.
- **Note** (*Note*) : évaluation d'un stagiaire pour une séance évaluable (CC ou EFM).
- **Avancement** (*Avancement*) : indicateur calculé, par couple Groupe/Module, du nombre d'heures réellement dispensées par rapport au volume horaire planifié.

2.1.3 Fonctionnalités attendues

- CRUD complet (**liste, détail, création, mise à jour, suppression**) pour les ressources qui le justifient métier (Affectation, Séance, Note), et lecture seule pour les ressources alimentées exclusivement par import Excel (Pôle, Espace, Formateur, Filière, etc.).
- Application des règles de gestion principales : intégrité référentielle entre entités, contraintes d'unicité (ex. un triplet Groupe/Module/Formateur/Mode unique), cohérence du pivot central Affectation.
- Consultation avancée par chargement à la demande des relations (paramètre `?include=a,b,c`), contrôlée par liste blanche par ressource.
- Filtrage, tri et recherche textuelle paramétrables sur chaque ressource, également contrôlés par liste blanche.
- Import en masse depuis des fichiers Excel/CSV existants, avec reconstruction automatique de la hiérarchie académique manquante et rapport d'erreurs ligne par ligne.

2.1.4 Acteurs

- *Application(s) client(s)* / front-office consommant l'API.
- *Administrateur métier* / coordination pédagogique, utilisateur de l'interface web d'administration.
- *Service scolarité*, producteur des fichiers Excel sources et utilisateur du module d'import.
- *Formateur* — acteur identifié pour une sécurisation future par service d'authentification (*externe*).

2.2 Cahier des charges technique (CDC-T)

2.2.1 Stack technique et contraintes

Élément	Choix retenu
Langage	PHP 8.3+
Framework	Laravel 13.x
Style d'architecture	API REST stateless, versionnée (/api/v1)
Base de données	PostgreSQL (prod / dev) — SQLite en mémoire pour les tests
Cache	Redis (cache applicatif + cache de réponses HTTP)

Élément	Choix retenu
Import de fichiers	PhpOffice/PhpSpreadsheet (Excel .xlsx/.xls et CSV)
Tests	PHPUnit (tests Feature et Unit), SQLite en mémoire
Frontal web léger	Blade + Vite + Tailwind CSS (interface d'administration interne)
Authentification	Non implémentée dans ce service d'api (système de data only)

2.2.2 Architecture applicative imposée

- **routes/api.php** : déclaration des routes versionnées sous le préfixe /v1.
- **app/Http/Controllers/Api/*Controller.php** : contrôleurs fins, sans logique métier.
- **app/Http/Requests/**/Store*Request.php** et **Update*Request.php** : validation des entrées.
- **app/Http/Requests/Filters/*FilterRequest.php** : validation stricte des paramètres de requête (filtres, tri, includes, pagination).
- **app/Filters/*Filter.php** : application déclarative des filtres sur le query builder Eloquent.
- **app/Services/*Service.php** : couche de service partagée entre l'API et le frontal web.
- **app/Http/Resources/*Resource.php** : sérialisation JSON des modèles.
- **app/Models/*** : modèles Eloquent représentant le domaine.

2.2.3 Contrat d'API (résumé)

- Endpoints au format `/api/v1/{ressource}` suivant la convention REST (apiResource).
- Pagination par paramètre `per_page` (borné à 100, valeur par défaut 15 à 20 selon la ressource).
- Chargement de relations via le paramètre `include` (liste CSV, intersectée avec une liste blanche par ressource).

2.2.4 Modèle de données — résumé de la hiérarchie

La hiérarchie générale du domaine s'organise ainsi :

Pôle → Espace / Formateur / Filière → Année → Groupe / Module → Stagiaire, avec le pivot Affectation = (Groupe + Module + Formateur), lui-même source des Séances (Affectation → Séance → Créneau horaire) et des Évaluations (Séance → Note ← Stagiaire). Ce modèle est détaillé au Chapitre 3.

2.2.5 Exigences non fonctionnelles

Catégorie	Exigence retenue
Performance	Chargement anticipé (eager-loading) des relations par défaut, includes contrôlés, pagination obligatoire, index de base de données sur les colonnes filtrées/triées.

Catégorie	Exigence retenue
Sécurité	API ouverte en phase 1 ; cible phase 2 : authentification, gestion des rôles. Dès la phase 1 : limitation de débit, protection contre les écritures intersites, journal d'audit, validation stricte de toutes les entrées.
Qualité	Tests PHPUnit (Feature/Unit), seeders et factories pour des données de test réalistes, journalisation applicative structurée.
Maintenabilité	Séparation stricte des responsabilités par couche, filtres et règles de validation déclaratifs et auto-documentés.

2.2.6 Critères d'acceptation

- La pagination est strictement bornée à 100 éléments maximum par page.
- Les paramètres include ignorent silencieusement tout chemin de relation non autorisé (et, en phase de validation amont, rejettent explicitement les chemins inconnus avec une erreur 422).
- L'unicité des affectations (triplet **Groupe/Module/Formateur + mode**) est garantie au niveau base de données.
- Les clés étrangères sont strictement validées avant toute écriture (existence en base).
- La suppression d'une entité respecte les règles de cascade ou de blocage définies dans le schéma (cascadeOnDelete / restrictOnDelete selon la relation).

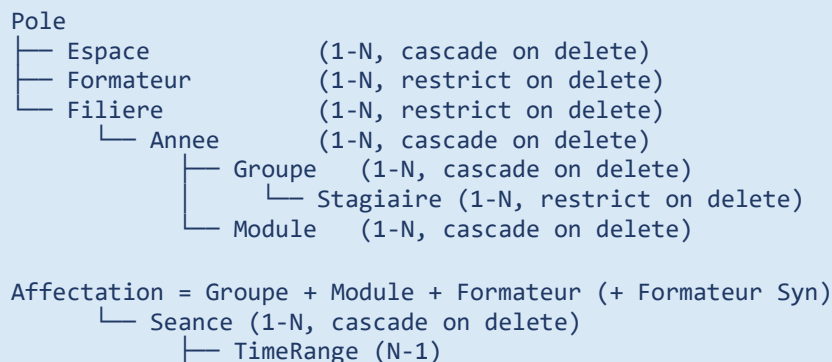
Chapitre 3 — Analyse et conception

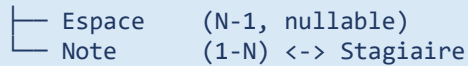
3.1 Modèle de données

La première étape du projet a consisté à analyser les fichiers Excel existants (Base_Formateurs.xlsx, AvancementProgramme*.xlsx, BasePlateEvaluation*.xlsx, lister_minimized.xlsx) afin d'en extraire un modèle entité-association normalisé. Plusieurs décisions de modélisation importantes ont émergé de cette analyse :

- Une Filière ne représente que le programme général de formation (ex. « Développement Digital ») ; toute option ou spécialisation de 2ème année (ex. « Option Développement Web Full Stack ») est portée par le libellé de l'entité Année, et non par une filière distincte. Ce choix évite une explosion du nombre de filières et reflète fidèlement la façon dont les options apparaissent réellement dans les fichiers sources.
- Certaines entités utilisent une clé primaire non auto-incrémentée correspondant à l'identifiant métier réel : Formateur (mle), Filière (code_filiere) et Stagiaire (cef). Cela permet de faire correspondre directement la clé primaire à l'identifiant utilisé dans les fichiers Excel sources et facilite l'opération d'upsert lors de l'import.
- Une Affectation relie un Groupe, un Module et un Formateur (le cas échéant un second formateur pour la partie synchrone), et porte les volumes horaires affectés (présentiel / synchrone). Une Séance ne peut exister que rattachée à une Affectation.
- Seules les séances de type « cc » (contrôle continu) ou « efm » (épreuve de fin de module) peuvent porter une Note ; une séance de type « cours » n'est jamais évaluable. Cette règle, identifiée via l'analyse des données réelles, est centralisée dans l'énumération NoteType (méthode evaluable()) et appliquée à trois niveaux : validation de la requête, validation applicative dans le service, et — pour les contraintes les plus critiques — au niveau de la base de données.
- Une Avancement (table avancements) est une entité dérivée, recalculée automatiquement à chaque création ou suppression de Séance, qui mesure le taux de réalisation horaire réel d'un module pour un groupe donné par rapport au volume horaire planifié du module.

Figure 1 — Hiérarchie générale des entités du domaine





Avancement = Groupe + Module (derivee, recalculee depuis les Seances)

3.1.1 Contraintes d'intégrité notables

Table	Contrainte
seances	Unicité (espace_id, date, time_range_id) — empêche la double réservation d'une salle.
affectations	Unicité (groupe_id, module_id, formateur_mle, mode) — une même combinaison ne peut être affectée deux fois pour le même mode.
notes	Unicité (seance_id, stagiaire_cef, type) — une seule note par stagiaire, séance et type.
groupes	Unicité (annee_id, code).
modules	Unicité (code_module, annee_id) — un même code de module peut se répéter d'une année à l'autre.
filieres -> niveaux / type_formations	restrictOnDelete — empêche la suppression d'un niveau ou d'un type de formation encore référencé.

3.2 Architecture logicielle

L'application suit une architecture en couches strictement séparées, organisée autour du framework Laravel, afin de garantir que la même logique métier (filtrage, pagination, chargement des relations) soit partagée entre les contrôleurs API et les contrôleurs de l'interface web d'administration, sans duplication.

Figure 2 — Flux de traitement d'une requête de lecture (GET /api/v1/{ressource})



```

- pagine le resultat
|
v
QueryFilter (ex. StagiaireFilter extends QueryFilter)
- traduit chaque parametre connu en clause Eloquent
|
v
Resource API (ex. StagiaireResource)
- serialise le modele (et ses relations chargees) en JSON
|
v
Reponse JSON paginee

```

3.2.1 Justification des choix d'architecture

- **Form Requests** dédiées au filtrage (**IndexFilterRequest**) : Laravel valide nativement le corps des requêtes d'écriture, mais ne fournit pas de mécanisme standard pour valider les paramètres de requête (query string) d'une liste. Une classe abstraite **IndexFilterRequest** a donc été créée pour mutualiser la validation de sort, include et per_page, et pour imposer le rejet explicite de tout paramètre non déclaré — un filtre inconnu silencieusement ignoré serait une source de bug difficile à détecter pour les consommateurs de l'API.
- **QueryFilter** déclaratif : plutôt que de multiplier les **conditions** **->when()** directement dans les contrôleurs, chaque ressource filtrable possède une classe **Filter** qui déclare une simple table de correspondance « paramètre → méthode ». Cela rend chaque filtre testable isolément, lisible et auto-documenté (le tableau **filters()** constitue la documentation du filtre).
- **BaseService** partagé entre API et Web : les contrôleurs Web (utilisés par l'interface d'administration Blade) et les contrôleurs API appellent exactement le même service, ce qui garantit que les deux interfaces appliquent rigoureusement les mêmes règles de filtrage, pagination et chargement de relations.
- **Resources** Laravel pour le contrat de sortie : chaque ressource API possède sa propre classe **Resource**, qui explicite la forme exacte du JSON renvoyé (y compris les relations chargées conditionnellement via **whenLoaded**), évitant ainsi toute fuite accidentelle de colonnes internes ou sensibles.

3.3 Choix technologiques

Besoin	Choix	Justification
Framework applicatif	Laravel 13 (PHP 8.3)	Écosystème mature, ORM Eloquent, validation intégrée, middleware, conventions REST.
Lecture/écriture Excel	PhpOffice/PhpSpreadsheet	Bibliothèque PHP de référence, gère .xlsx/.xls/.csv sans dépendance externe.
Cache applicatif	Redis	Cache des réponses HTTP de lecture,

Besoin	Choix	Justification
		verrouillage anti-stampede, invalidation par tags.
Base de données	PostgreSQL	SGBD relationnel robuste, contraintes d'intégrité avancées (CHECK), bon support Laravel.
Tests	PHPUnit + SQLite mémoire	Exécution rapide et isolée des tests, sans dépendance à une base externe.
Frontal d'administration	Blade + Tailwind CSS + Vite	Interface interne légère, sans framework JS lourd, réutilisant les mêmes services métier.

Chapitre 4 — Routes, requêtes et réponses de l'API

Ce chapitre constitue le cœur du contrat d'API : il décrit exhaustivement la structure des routes exposées, le format attendu des requêtes (paramètres, corps, règles de validation), le format des réponses produites, ainsi que le système de filtrage avancé qui répond directement à la problématique initiale (impossibilité de filtrer les données dans les fichiers Excel).

4.1 Structure générale des routes

Toutes les routes de l'API sont déclarées dans `routes/api.php` et préfixées par `/api/v1`, conformément à une logique de versionnage explicite qui permettra de faire évoluer le contrat sans casser les clients existants. Les ressources se répartissent en deux catégories, selon que la donnée est uniquement importée depuis Excel (lecture seule) ou gérée directement via l'API (CRUD complet).

4.1.1 Ressources en lecture seule (index + show)

Ces ressources sont exclusivement alimentées par le module d'import Excel décrit au Chapitre 5. Exposer des routes `store/update/destroy` sur ces contrôleurs aurait été trompeur (le contrôleur ne les implémente pas), c'est pourquoi seules les actions `index` et `show` sont enregistrées via `->only(['index', 'show'])`.

Ressource	Verbe / Route	Description
poles	GET <code>/api/v1/poles</code>	Liste paginée et filtrable des pôles
<code>poles/{pole}</code>	GET <code>/api/v1/poles/{id}</code>	Détail d'un pôle
espaces	GET <code>/api/v1/espaces/{id}</code>	Salles, laboratoires, amphithéâtres
formateurs	GET <code>/api/v1/formateurs/{mle}</code>	Formateurs (clé = matricule)
niveaux	GET <code>/api/v1/niveaux/{id}</code>	Niveaux de qualification
type-formations	GET <code>/api/v1/type-formations/{id}</code>	Types de formation
filieres	GET <code>/api/v1/filieres/{code}</code>	Filières (clé = <code>code_filiere</code>)
annees	GET <code>/api/v1/annees/{id}</code>	Années de formation (avec option)
modules	GET <code>/api/v1/modules/{id}</code>	Modules d'enseignement
stagiaires	GET <code>/api/v1/stagiaires/{cef}</code>	Stagiaires (clé = CEF)
time-ranges	GET <code>/api/v1/time-ranges/{id}</code>	Créneaux horaires
avancements	GET <code>/api/v1/avancements/{id}</code>	Avancement pédagogique calculé
groupes	GET <code>/api/v1/groupes/{id}</code>	Groupes de stagiaires

4.1.2 Ressources en CRUD complet

Ces trois ressources implémentent les cinq actions REST standard via `Route::apiResource()`, car elles correspondent à des objets métier que les utilisateurs (coordination pédagogique) doivent pouvoir créer, corriger ou supprimer directement, indépendamment de tout import Excel.

Ressource	Verbe	Route	Action
affectations	GET	/api/v1/affectations	index — liste filtrable
	GET	/api/v1/affectations/{id}	show — détail
	POST	/api/v1/affectations	store — création
	PUT/PATCH	/api/v1/affectations/{id}	update — mise à jour
	DELETE	/api/v1/affectations/{id}	destroy — suppression
seances	GET/POST/PUT/PATCH/DELETE	/api/v1/seances/{id}	CRUD complet
notes	GET/POST/PUT/PATCH/DELETE	/api/v1/notes/{id}	CRUD complet

4.1.3 Routes d'agrégation hiérarchique (« cascade »)

Un sous-groupe de routes dédiées, préfixé par `/api/v1/hierarchy`, a été ajouté pour répondre à un besoin précis des interfaces clientes : permettre la construction de menus déroulants en cascade (sélection d'un pôle → filtrage des filières → filtrage des années → filtrage des groupes, etc.) sans avoir à connaître à l'avance les paramètres de filtrage génériques de chaque ressource. Ces routes renvoient des réponses JSON allégées, optimisées pour ce cas d'usage précis.

Route	Paramètre(s) optionnel(s)	Usage
GET /hierarchy/poles	—	Liste de tous les pôles (id, libellé)
GET /hierarchy/filieres	pole_id	Filières d'un pôle
GET /hierarchy/annees	filiere_code ou pole_id	Années d'une filière (ou d'un pôle)
GET /hierarchy/groupes	annee_id, filiere_code ou pole_id	Groupes correspondants
GET /hierarchy/modules	annee_id, groupe_id ou filiere_code	Modules correspondants
GET /hierarchy/stagiaires	groupe_id, q (recherche)	Stagiaires d'un groupe (recherche limitée à 50 résultats si q seul)
GET /hierarchy/seances	groupe_id, module_id, date	Séances filtrées
GET /hierarchy/niveaux	—	Liste des niveaux
GET /hierarchy/resolve/stagiaire/{cef}	—	Résout un CEF en sa hiérarchie complète (groupe, année, filière, pôle)

Route	Paramètre(s) optionnel(s)	Usage
GET /hierarchy/resolve/groupe/{id}	—	Résout un groupe en sa hiérarchie complète (année, filière, pôle)

4.1.4 Route d'import

POST /api/v1/import

Détaillée au Chapitre 5, cette route accepte un envoi multipart/form-data contenant un fichier Excel/CSV et importe son contenu selon le type demandé (formateurs ou stagiaires).

4.2 Format des requêtes

Toute requête d'écriture (création ou mise à jour) est interceptée par une classe Form Request dédiée (Store*Request ou Update*Request) qui définit, de façon déclarative, l'ensemble des règles de validation Laravel applicables. Cette classe est injectée directement dans la signature du contrôleur ; si la validation échoue, Laravel court-circuite automatiquement l'exécution et renvoie une réponse 422 avant que le contrôleur ne soit atteint.

4.2.1 Exemple — création d'une séance (StoreSeanceRequest)

Cet exemple illustre plusieurs techniques de validation utilisées dans le projet : validation de présence et de type, validation contre une liste de valeurs autorisées, validation référentielle (exists:), et règle de validation personnalisée (closure) implémentant une vérification métier (non-double réservation d'une salle).

POST /api/v1/seances
Content-Type: application/json

```
{
  "affectation_id": 12,
  "type": "cc",
  "date": "2026-03-10",
  "time_range_id": 2,
  "espace_id": 5
}
```

Champ	Règles	Remarque
affectation_id	required, integer, exists:affectations,id	L'affectation doit exister.
type	required, string, in:cours,cc,efm,exam	Typage strict de la séance.
date	required, date, date_format:Y-m-d	Format ISO imposé.
time_range_id	required, integer, exists:time_ranges,id	Créneau horaire référencé.
espace_id	nullable, integer, exists:espaces,id, + règle métier	Nul si séance en ligne ; sinon vérifie la disponibilité de la salle pour la date et le créneau (méthode Espace::isAvailable()).

En cas de double réservation, l'API renvoie une erreur 422 avec un message explicite : « Cet espace est déjà réservé pour cette date et ce créneau horaire. » Ce contrôle est exécuté avant toute écriture en base, évitant ainsi de s'appuyer uniquement sur la contrainte d'unicité de la base de données (qui existe également, en défense en profondeur).

4.2.2 Format des requêtes de mise à jour (sémantique PATCH)

Les requêtes de mise à jour (Update*Request) utilisent systématiquement la règle `sometimes` plutôt que `required` : seuls les champs effectivement transmis sont validés et modifiés, ce qui permet au client de n'envoyer que les champs qu'il souhaite réellement changer, conformément à la sémantique PATCH.

```
PATCH /api/v1/notes/{id}
Content-Type: application/json
```

```
{
  "valeur": 14.5,
  "decision": "Admis"
}
```

4.2.3 Validation des paramètres de requête (filtres, tri, pagination)

Le second grand type de requête est la requête de liste (GET /api/v1/{ressource}). Pour ce type de requête, c'est la chaîne de requête (query string) qui doit être validée, et non un corps JSON. Une classe abstraite commune, `IndexFilterRequest`, factorise cette validation pour l'ensemble des ressources filtrables et impose quatre garanties :

7. Tout paramètre de filtre déclaré par la ressource (via `filterRules()`) est validé selon des règles Laravel classiques (type, plage, appartenance à une liste).
8. Le paramètre `sort` est validé contre la liste exacte des colonnes triables de la ressource (`sortable()`), avec prise en charge de la syntaxe « -colonne » pour le tri décroissant.
9. Le paramètre `include` est validé contre la liste exacte des relations chargeables (`allowedIncludes()`), avec une profondeur maximale de 4 niveaux et un maximum de 10 chemins simultanés, afin de prévenir les abus de type N+1 ou épuisement mémoire.
10. Tout paramètre de requête non reconnu déclenche une erreur 422 explicite listant les paramètres autorisés, plutôt que d'être silencieusement ignoré.

Exemple de réponse d'erreur lorsqu'un paramètre inconnu est transmis :

```
GET /api/v1/stagiaires?genre=X&groupe=ABC
```

```
HTTP/1.1 422 Unprocessable Entity
```

```
{
  "message": "Invalid filter/query parameters.",
  "errors": {
    "genre": ["The genre field contains an invalid value: \"X\". Allowed: F, H."],
    "query": ["Unsupported query parameter(s): groupe. Allowed parameters: q, nom, prenom, cef, dni, genre, actif, groupe_id, groupe_code, annee_id, filiere_code,
```

```
pole_id, niveau_id, age_min, age_max, date_naissance_from, date_naissance_to,
per_page, page, sort, include." ]
}
}
```

4.2.4 Règles de validation personnalisées (defense in depth)

Pour les filtres acceptant une liste de valeurs séparées par des virgules (ex. genre=F,H ou groupe_id=1,2,3), trois règles de validation personnalisées ont été développées afin de contenir des risques précis identifiés lors de la revue de sécurité du projet :

- **CsvOfIntegers** : vérifie que chaque élément est un entier strictement positif et borne le nombre d'éléments (par défaut 100). Bien qu'Eloquent paramètre déjà chaque valeur d'un `whereIn()` (donc sans risque d'injection SQL), cette règle évite qu'un client envoie des milliers de valeurs dans une clause `IN()`, ce qui dégraderait les performances (dénégation de service applicatif).
- **CsvOfStrings** : même principe pour les colonnes texte libres (code_filiere, code_module, secteur...), avec une borne sur la longueur de chaque élément et sur le nombre total d'éléments.
- **CsvOfEnum** : restreint chaque élément à un ensemble fermé de valeurs autorisées (ex. statut du formateur, décision de la note, type de séance) — la véritable barrière de sécurité pour ces colonnes, puisqu'elle documente et impose le vocabulaire exact accepté par l'API.

4.3 Format des réponses

4.3.1 Réponse d'une collection paginée

Toute réponse de liste suit le format standard de pagination de Laravel (`LengthAwarePaginator`), enrichi par la classe `Resource` correspondant à la ressource. La pagination est bornée côté serveur entre 1 et 100 éléments (paramètre `per_page`), quelle que soit la valeur demandée par le client.

```
GET /api/v1/stagiaires?genre=F&actif=1&per_page=2
```

```
HTTP/1.1 200 OK
Content-Type: application/json
```

```
{
  "data": [
    {
      "cef": "2006091400263",
      "groupe_id": 3,
      "nom": "AOULAD BRAHIM",
      "prenom": "IKRAM",
      "nom_complet": "AOULAD BRAHIM IKRAM",
      "genre": "F",
      "actif": true,
      "created_at": "2026-04-15T10:00:00.000000Z",
    }
  ]
}
```

```

    "updated_at": "2026-04-15T10:00:00.000000Z",
    "groupe": []
  },
  "links": { "first": "...", "last": "...", "prev": null, "next": "..." },
  "meta": {
    "current_page": 1,
    "per_page": 2,
    "last_page": 7,
    "total": 14
  }
}

```

Remarque : les champs sensibles `cni`, `telephone`, `adresse` et `date_naissance` du modèle `Stagiaire` sont déclarés `$hidden` au niveau du modèle `Eloquent`, en complément du contrôle explicite réalisé dans `StagiaireResource` — une double protection contre toute fuite accidentelle d'information personnelle.

4.3.2 Relations chargées à la demande (?include=)

Par défaut, une ressource ne renvoie pas ses relations (pour limiter la taille de la réponse). Le client peut demander explicitement le chargement d'une ou plusieurs relations via le paramètre `include`, sous réserve qu'elles figurent dans la liste blanche définie par la ressource.

```

GET /api/v1/stagiaires/2006091400263?include=groupe,groupe.annee.filiere,notes.seance

HTTP/1.1 200 OK
{
  "data": {
    "cef": "2006091400263",
    ...
    "groupe": {
      "id": 3,
      "code": "DEV103",
      "annee": { "id": 1, "libelle": "1\u00e8re ann\u00e9e - Tronc Commun",
        "filiere": { "code_filiere": "DIA_DEV_TS", "libelle":
"D\u00e9veloppement Digital" } }
    },
    "notes": [ { "id": 41, "valeur": "14.50", "decision": "Admis",
      "seance": { "id": 7, "type": "cc", "date": "2026-02-12" } } ]
  }
}

```

4.3.3 Codes de statut HTTP utilisés

Code	Signification	Cas d'usage dans le projet
200 OK	Succès	index, show, update (corps JSON)
201 Created	Création réussie	store (Affectation, Séance, Note)
204 No Content	Suppression réussie	destroy
207 Multi-	Succès partiel	Import Excel : certaines lignes ont échoué (voir

Code	Signification	Cas d'usage dans le projet
Status		Chapitre 5)
403 Forbidden	Origine non fiable	Écriture intersite bloquée (RejectSuspiciousCrossSiteRequests)
422 Unprocessable Entity	Validation échouée	Form Request invalide, filtre/tri/include inconnu, jeton CSRF invalide
429 Too Many Requests	Limite de débit atteinte	RateLimiter (voir Chapitre 5)

Toutes les réponses, y compris les erreurs, portent l'en-tête X-Request-Id (voir 5.3.1), permettant de corréler une réponse côté client avec l'entrée correspondante dans le journal d'audit serveur.

4.4 Système de filtrage avancé

Le système de filtrage est la réponse directe à la problématique initiale du stage : permettre des recherches structurées et combinables que les fichiers Excel ne permettaient pas. Chaque ressource filtrable expose une classe Filter (héritant de QueryFilter) qui déclare, sous forme de tableau associatif, l'ensemble des paramètres de requête supportés et la méthode chargée de les traduire en clause Eloquent.

4.4.1 Principe de fonctionnement de QueryFilter

```
abstract class QueryFilter {
    abstract protected function filters(): array; // param => méthode

    public function apply(Builder $builder): Builder {
        foreach ($this->filters() as $param => $method) {
            if (! $this->request->filled($param)) continue;
            if (! method_exists($this, $method)) continue;
            $this->{$method}($this->request->input($param));
        }
        $this->applySort();
        return $this->builder;
    }
}
```

Ce mécanisme garantit qu'un paramètre non déclaré dans filters() n'a strictement aucun effet sur la requête SQL générée — il ne pourrait donc en aucun cas être exploité pour injecter une condition arbitraire, même en cas d'oubli côté validation. C'est la première barrière de sécurité du système de filtrage ; la seconde, plus stricte, est la validation explicite réalisée en amont par le Form Request (qui rejette le paramètre inconnu avant même d'atteindre le filtre).

4.4.2 Exemple détaillé — filtrage des stagiaires

La ressource Stagiaire illustre le filtrage le plus riche du projet, avec 17 paramètres combinables : recherche libre, filtres exacts, filtres partiels, filtres par liste, filtres relationnels (remontant la hiérarchie via whereHas) et filtres calculés (âge dérivé de la date de naissance).

Paramètre	Comportement	Exemple
q	Recherche libre sur nom, prénom, CEF, CNI	?q=alami
nom / prenom	Correspondance partielle (LIKE)	?nom=ALAMI
cef / cni	Correspondance exacte	?cef=2006091400263
genre	Liste fermée (F, H), max 2 valeurs	?genre=F
actif	Booléen (accepte 1/0/oui/non/true/false)	?actif=1
groupe_id	Liste d'entiers (max 100)	?groupe_id=3,4,5
groupe_code	Correspondance partielle sur le code du groupe (jointure)	?groupe_code=DEV1
annee_id / filiere_code / pole_id / niveau_id	Remontée de la hiérarchie via whereHas() en cascade	?pole_id=2
age_min / age_max	Calcul dérivé sur date_naissance	?age_min=18&age_max=25
date_naissance_from / date_naissance_to	Plage de dates	?date_naissance_from=2005-01-01
sort	Tri (préfixe « - » = décroissant)	?sort=-cef
include	Relations à charger (liste blanche)	?include=groupe,notes
per_page / page	Pagination bornée à 100	?per_page=50&page=2

Exemple combiné — « tous les stagiaires actifs, de genre féminin, du pôle 2, âgés de 18 à 25 ans, triés par nom, avec leur groupe » :

```
GET
/api/v1/stagiaires?actif=1&genre=F&pole_id=2&age_min=18&age_max=25&sort=nom&include=groupe
```

Une recherche équivalente sur les fichiers Excel sources aurait nécessité l'application manuelle et successive de plusieurs filtres natifs, sans garantie de reproductibilité ni de fiabilité — c'est précisément ce manque que ce système comble.

4.4.3 Filtres relationnels remontant la hiérarchie

Un point d'attention particulier a été porté aux filtres permettant de remonter la hiérarchie académique depuis une ressource « feuille ». Par exemple, filtrer les séances par pôle nécessite de traverser quatre niveaux de relations (Seance → Affectation → Groupe → Année → Filière → Pôle) :

```
protected function filterPoleId(mixed $value): void {
    $this->builder->whereHas(
        'affectation.groupe.annee.filiere',
        fn ($q) => $q->where('pole_id', $value)
    );
}
```


Ce type de filtre est disponible, selon la ressource, sur : Séance, Note, Module, Affectation et Stagiaire, ce qui permet par exemple d'obtenir directement « toutes les notes manquantes du pôle X » ou « toutes les séances du pôle X programmées cette semaine » sans avoir à enchaîner plusieurs appels d'API côté client.

4.4.4 Tableau récapitulatif des filtres disponibles par ressource

Ressource	Filtres clés	Tri (sort)
Pole	q, libelle, has_formateurs, has_filiere, has_espaces	libelle, created_at
Filiere	q, code_filiere, libelle, pole_id, niveau_id, type_formation_id, secteur	libelle, code_filiere, created_at
Annee	filiere_code, libelle, pole_id	libelle, created_at
Groupe	q, code, annee_id, mode, effectif_min/max, filiere_code, pole_id, niveau_id, has_stagiaires	code, effectif, created_at
Module	q, code_module, libelle, annee_id, regional, filiere_code, pole_id, has_affectations	libelle, code_module, created_at
Formateur	q, mle, nom_prenom, statut, pole_id, efp_mutualise, mutualise, mhs_min/max, has_affectations, email_edu	nom_prenom, mle, mhs, statut, created_at
Stagiaire	(voir 4.4.2 — 17 paramètres)	cef, nom, prenom, date_naissance, created_at
Affectation	groupe_id, module_id, module_code, formateur_mle, mode, mh_affecte_min/max, mh_totale_min/max, pole_id, filiere_code, has_seances	mh_affecte, mh_affecte_syn, mode, created_at
Seance	type, date, date_from/to, espace_id, time_range_id, affectation_id, groupe_id, module_code, formateur_mle, pole_id, filiere_code, annee_id, has_notes, weekday	date, created_at
Note	seance_id, stagiaire_cef, type, decision, valeur_min/max, passing, missing, groupe_id, pole_id, filiere_code, annee_id	valeur, created_at
Espace	q, libelle, pole_id, capacite_min/max, effectif, available_date + available_time_range_id	libelle, capacite, created_at
Avancement	groupe_id, module_id, filiere_code, termine	groupe_id, module_id, mh_realisee_globale, taux_realisation_globale, created_at

Le filtre **missing=1** sur la ressource Note est un exemple typique de valeur ajoutée directe du projet : il identifie en une seule requête toutes les évaluations restant à saisir (valeur IS NULL), une opération qui demandait auparavant un parcours manuel des fichiers Excel pour repérer les cellules vides.

4.5 Gestion des erreurs

Le projet adopte une politique homogène de remontée d'erreur, fondée sur les exceptions natives de Laravel converties en réponses JSON structurées :

- Erreurs de validation (FormRequest, IndexFilterRequest) : **422**, avec un objet errors détaillant chaque champ en cause.
- Ressource introuvable (findOrFail, route model binding) : **404**, gérée nativement par Laravel.
- Origine de requête non fiable (écriture intersite) : **403**, gérée par le middleware RejectSuspiciousCrossSiteRequests avec un message explicite et l'identifiant de corrélation.
- Jeton CSRF invalide (TokenMismatchException) ou origine non reconnue (OriginMismatchException) : **422 / 403**, interceptées par des gestionnaires d'exception dédiés déclarés dans bootstrap/app.php pour produire un JSON cohérent plutôt que la page d'erreur HTML par défaut de Laravel.
- Dépassement de quota (RateLimiter) : **429**, avec les en-têtes standard Retry-After et X-RateLimit-*.

Chapitre 5 — Réalisation technique

5.1 Module d'import Excel / CSV

Le module d'import (ImportService, exposé via POST /api/v1/import) constitue la passerelle entre les fichiers Excel historiques du centre et la nouvelle source de vérité. Il a été conçu pour tolérer l'hétérogénéité réelle des fichiers rencontrés plutôt que d'imposer un format rigide aux utilisateurs.

5.1.1 Format de la requête d'import

```
POST /api/v1/import
Content-Type: multipart/form-data

file : fichier .xlsx | .xls | .csv (max 20 Mo)
type : "formateurs" | "stagiaires"
mode : "upsert" (default) | "insert_only"
```

Paramètre	Règles de validation	Comportement
file	required, file, mimes:xlsx,xls,csv, max:20480	Rejette tout autre format ou fichier trop volumineux.
type	required, in:formateurs,stagiaires	Détermine le mappage de colonnes utilisé.
mode	sometimes, in:upsert,insert_only	upsert : crée ou met à jour ; insert_only : ignore les lignes dont la clé existe déjà.

5.1.2 Normalisation des en-têtes de colonnes

Les en-têtes des fichiers Excel produits par le centre varient fortement (accents, espaces, casse, abréviations). Une fonction de normalisation transforme chaque en-tête en un identifiant interne stable : passage en minuscules, suppression des accents français courants, remplacement de toute séquence non alphanumérique par un underscore. Ainsi, « Nom et Prénom », « NOM_ET_PRENOM » ou « Nom Et Prénom » sont tous reconnus comme la même colonne logique nom_et_prenom. Pour chaque champ métier, plusieurs alias de colonnes possibles sont tentés dans l'ordre (méthode cell()), ce qui permet au service de s'adapter à plusieurs variantes de fichiers (export « formateurs » classique ou export de type rapport d'avancement) sans configuration supplémentaire.

5.1.3 Reconstruction automatique de la hiérarchie académique

L'apport principal du module d'import est sa capacité à reconstruire automatiquement, ligne par ligne, l'ensemble de la chaîne hiérarchique Pôle → Niveau → Type de formation → Filière → Année → Groupe à partir des seules colonnes plates d'un export stagiaire, en utilisant systématiquement firstOrCreate() pour éviter toute duplication : un pôle, une filière ou un groupe déjà créé par une ligne précédente (ou par un import antérieur) est réutilisé plutôt que recréé.

Figure 3 — Résolution de la hiérarchie lors de l'import d'un stagiaire

```

Ligne Excel (stagiaire)
|
+--> Pole          firstOrCreate(libelle = Secteur)
+--> Niveau        firstOrCreate(libelle = Niveau)
+--> TypeFormation firstOrCreate(libelle = Type de formation)
+--> Filiere        firstOrCreate(code_filiere = Code_I, [libelle, pole_id,
niveau_id, ...])
+--> Annee          firstOrCreate(filiere_code, libelle = annee deduite du code
groupe)
+--> Groupe         firstOrCreate(code = GROUP, [annee_id, ...])
+--> Stagiaire      updateOrCreate(cef, [groupe_id, nom, prenom, ...])

```

L'année de formation (1ère ou 2ème année) est déduite automatiquement du code du groupe lorsque l'information n'est pas explicitement fournie, via une expression régulière reconnaissant les motifs habituels (ex. « DEV101 » → 1, « DEVOWFS201 » → 2).

5.1.4 Tolérance aux erreurs ligne par ligne

Conformément à un des objectifs du cahier des charges, l'import ne doit pas échouer intégralement à cause d'une seule ligne défectueuse (par exemple un CEF manquant). Chaque ligne est traitée dans son propre bloc try/catch : une erreur métier (InvalidArgumentException) est collectée dans un tableau d'erreurs avec le numéro de ligne et un aperçu sécurisé de la ligne, sans interrompre le traitement des lignes suivantes. L'ensemble de l'import s'exécute néanmoins dans une transaction de base de données unique (DB::transaction), garantissant qu'en cas d'erreur technique imprévue (et non d'une simple erreur métier de ligne), aucune écriture partielle ne subsiste.

Le code de statut HTTP de la réponse reflète ce résultat mixte : 200 si toutes les lignes ont été traitées sans erreur, 207 (Multi-Status) dès qu'au moins une ligne a échoué — l'appelant peut alors consulter le tableau row_errors pour corriger le fichier source et relancer l'import en mode insert_only afin de ne traiter que les lignes manquantes.

```

HTTP/1.1 207 Multi-Status
{
  "success": true,
  "type": "stagiaires",
  "mode": "upsert",
  "summary": { "inserted": 27, "updated": 0, "skipped": 0, "errors": 1 },
  "row_errors": [
    { "row": 14, "message": "Missing required field: CEF (matricule \u00e9tudiant).",
      "data": { "cef": "", "nom": "ALAMI", "group": "DEV101" } }
  ]
}

```

5.2 Sécurité

Bien que l'API soit volontairement ouverte (sans authentification) à ce stade du projet, plusieurs mécanismes de sécurité transverses ont été mis en place dès la phase 1, conformément aux exigences non fonctionnelles du cahier des charges.

5.2.1 Identifiant de corrélation (Request ID)

Le middleware `AssignRequestId` attribue à chaque requête entrante un identifiant unique (en-tête `X-Request-Id`, fourni par le client ou généré sous forme d'UUID), restitué dans l'en-tête de la réponse et utilisé dans tous les messages d'erreur et entrées de journal. Cela permet de relier précisément un incident signalé côté client à la trace correspondante côté serveur.

5.2.2 Protection contre les écritures intersites

Le middleware `RejectSuspiciousCrossSiteRequests` bloque toute requête d'écriture (POST/PUT/PATCH/DELETE) dont l'origine HTTP n'est pas reconnue comme fiable. Il combine plusieurs signaux pour décider du blocage :

- L'en-tête `Origin` (ou, à défaut, l'origine déduite du `Referer`) doit appartenir à la liste des origines fiables configurées (**`SECURITY_TRUSTED_ORIGINS`** / **`CORS_ALLOWED_ORIGINS`**).
- L'en-tête `Sec-Fetch-Site`, lorsqu'il vaut `cross-site`, déclenche un blocage systématique si l'origine n'est pas fiable.
- Le couple CORS (`config/cors.php`) restreint, en complément, les méthodes, en-têtes et origines autorisés au niveau navigateur.

Chaque blocage est journalisé (canal d'audit) avec le motif précis (`untrusted_origin`, `cross_site_write`, `same_site_write_disabled`, `missing_origin`).

5.2.3 Limitation de débit (rate limiting)

Le `RateLimiter` Laravel est configuré dans `AppServiceProvider` avec des quotas distincts selon que la requête est une lecture ou une écriture, et selon la route précise appelée — afin qu'un client ne puisse pas saturer une route particulière même s'il reste sous le quota global.

Type de requête	Quota / minute	Quota / heure	Quota par route / minute
Lecture (GET/HEAD)	120	1500	60
Écriture (POST/PUT/PATCH/DELETE)	30	300	60
Interface web (Blade)	180 (global)	—	—

La clé de quota (`SecurityContext::clientKey`) combine, par ordre de priorité, l'utilisateur authentifié (si présent), le jeton bearer, la clé API (en-tête `X-API-Key`), ou à défaut une empreinte HMAC de l'adresse IP — ce qui anticipe directement l'arrivée de l'authentification en phase 2 sans nécessiter de réécriture de cette logique.

5.2.4 Journal d'audit

Le middleware `AuditRequests` journalise, pour chaque requête d'écriture (et chaque requête en échec, configurable), un enregistrement structuré au format JSON (canal audit, rotation quotidienne, rétention 90 jours) comprenant l'identifiant de requête, la route, le client, le code de

statut, la durée, et — sur option — les champs d'entrée assainis (avec masquage automatique des champs sensibles : mots de passe, jetons).

5.2.5 Validation stricte de toutes les entrées

Au-delà des middlewares, la sécurité repose en premier lieu sur la stratégie de validation décrite au Chapitre 4 : tout paramètre non déclaré est rejeté, toute valeur de liste fermée (enum) est vérifiée, toute longueur ou cardinalité de liste est bornée. Cette approche « liste blanche systématique » constitue la défense principale contre les abus de filtrage et de tri (impossibilité d'injecter une colonne ou une condition SQL arbitraire via les paramètres de requête).

5.3 Performance et mise en cache

Le middleware `CacheDataResponses` met en cache, dans Redis, le corps et les en-têtes pertinents des réponses de lecture (statuts 200), avec une durée de vie configurable (`DATA_CACHE_TTL`, 600 secondes par défaut). Plusieurs précautions ont été prises pour garantir la fraîcheur des données :

- La clé de cache intègre la méthode, l'URL complète, la route, l'en-tête `Accept` et l'identité de l'appelant — deux requêtes différant uniquement par un paramètre de filtre obtiennent donc des entrées de cache distinctes.
- Toute requête d'écriture qui aboutit (statut < 400) déclenche un vidage immédiat (flush) du cache de données, garantissant qu'aucune réponse obsolète ne soit servie après une modification.
- Un verrou (lock) Redis empêche l'effet de horde (« cache stampede ») : si plusieurs requêtes identiques arrivent simultanément alors que l'entrée n'est pas encore en cache, une seule exécute réellement la requête, les autres attendent puis profitent du résultat fraîchement calculé.
- En cas d'indisponibilité du cache ou de dépassement du délai de verrouillage, la requête est exécutée normalement (mode dégradé, en-tête `X-Data-Cache: BYPASS`) plutôt que de faire échouer la requête.

L'en-tête de réponse `X-Data-Cache` (HIT / MISS / BYPASS) permet de diagnostiquer rapidement, y compris en production, si une réponse provient du cache ou d'un calcul effectif.

5.4 Interface web d'administration

En complément de l'API REST, une interface web légère (Blade + Tailwind CSS) a été développée pour la coordination pédagogique, réutilisant exactement les mêmes services métier (`BaseService` et ses dérivés) que l'API. Cela garantit une cohérence stricte entre ce que voit un administrateur dans le navigateur et ce que renvoie l'API à un consommateur tiers, et évite toute duplication des règles de filtrage et de pagination. L'interface propose notamment un tableau de bord de synthèse (`DashboardController`) agrégeant les indicateurs clés du centre (effectifs, séances récentes, notes manquantes) ainsi qu'un écran d'import de fichiers Excel/CSV consommant la même route `POST /api/v1/import` que l'API.

Chapitre 6 — Tests et validation

La fiabilité du module d'import et des middlewares de sécurité étant critique pour la confiance accordée à l'API en tant que source de vérité, une stratégie de tests automatisés a été mise en place dès le début du projet, à l'aide de PHPUnit et d'une base de données SQLite en mémoire (RefreshDatabase), garantissant des tests rapides, isolés et reproductibles.

6.1 Tests fonctionnels (Feature)

Suite de tests	Couverture
ImportControllerTest	Validation des champs requis, rejet d'un type d'import invalide, import réussi de formateurs et de stagiaires avec vérification de la hiérarchie créée, comportement upsert vs insert_only, gestion des lignes en erreur (statut 207), non-duplication d'un pôle partagé entre plusieurs lignes.
SecurityMiddlewareTest	Préflight CORS pour une origine autorisée, blocage d'une écriture intersite depuis une origine non fiable, présence de l'en-tête X-Request-Id en sortie, réponse 429 structurée en cas de dépassement de quota.
DataCacheMiddlewareTest	Mise en cache effective d'une réponse de lecture (MISS puis HIT), invalidation du cache après une écriture réussie sur la même route.

6.2 Données de test réalistes (factories et seeders)

Pour fiabiliser les tests d'intégration et disposer d'un environnement de démonstration représentatif, des factories Eloquent ont été développées pour chaque entité, avec des états (states) reflétant des cas métier réels observés dans les fichiers sources (ex. AffectationFactory::coherentAnnee() garantissant qu'un module affecté à un groupe appartient bien à la même année que ce groupe). Des seeders dédiés (FormateursSeeder, StagiairesSeeder) rejouent un sous-ensemble anonymisé mais structurellement fidèle des données réelles du Pôle Digital et IA, ordonnancés dans DatabaseSeeder selon le graphe de dépendances du schéma (référentiels → structure académique → personnes → planification → évaluations).

6.3 Exemple de scénario de test

```
public function test_imports_stagiaires_successfully(): void
{
    $file = $this->makeXlsx([
        ['CEF', 'NOM', 'GROUP', 'FILIERE', 'NIVEAU', 'SECTEU', ...],
        ['2006091400263', 'ALAMI', 'DEV101', 'Dev Web FS', 'TS', 'Pôle
Numerique', ...],
    ]);

    $response = $this->post('/api/v1/import', ['file' => $file, 'type' =>
'stagiaires']);
}
```

```
$response->assertStatus(200)
    ->assertJsonPath('summary.inserted', 2)
    ->assertJsonPath('summary.errors', 0);

$this->assertDatabaseHas('stagiaires', ['cef' => '2006091400263']);
$this->assertDatabaseHas('filieres',    ['code_filiere' => 'DEV_TS']);
$this->assertDatabaseHas('groupes',     ['code' => 'DEV101']);
}
```

Ce test illustre la double vérification systématiquement appliquée dans le projet : vérification du contrat HTTP (statut, structure JSON) et vérification de l'état réel de la base de données après l'opération — garantissant que l'API ne se contente pas de répondre correctement, mais produit effectivement l'effet attendu.

Chapitre 7 — Difficultés rencontrées et solutions apportées

Difficulté rencontrée	Solution apportée
Incohérence des en-têtes de colonnes entre les différents fichiers Excel du centre (accents, espaces, abréviations différentes selon l'export).	Fonction de normalisation des en-têtes (suppression d'accents, minuscules, snake_case) combinée à une résolution par liste d'alias candidats pour chaque champ métier (méthode cell()).
Une filière (« Développement Digital ») apparaissait dans les fichiers sources tantôt comme entité générale, tantôt fusionnée avec son option de spécialisation, rendant ambigu le choix de modélisation.	Décision de modélisation explicite : la Filière reste générale ; l'option est portée par le libellé de l'Année. Ce choix a été documenté dans le modèle (commentaires de classe) afin d'éviter toute régression future.
Risque qu'un import partiel laisse la base de données dans un état incohérent (par exemple un stagiaire créé sans groupe valide) en cas d'erreur en cours de traitement.	Encapsulation de l'ensemble de l'import dans une transaction de base de données unique, combinée à une gestion d'erreur ligne par ligne qui ne fait pas échouer la transaction globale pour une simple erreur métier (distinction entre exception métier et exception technique).
Filtrer une ressource « feuille » (ex. Note) selon un critère situé plusieurs niveaux plus haut dans la hiérarchie (ex. le pôle) nécessitait de traverser quatre relations successives, au risque de générer des requêtes SQL coûteuses (N+1).	Utilisation systématique de whereHas() imbriqués pour traduire ces filtres en une seule requête SQL avec sous-requêtes corrélées, plutôt que de charger les relations puis de filtrer en mémoire PHP.
Un paramètre de filtre mal orthographié ou inexistant (ex. ?genree=F) pouvait être silencieusement ignoré par le système de filtrage initial, produisant un résultat non filtré sans que le client en soit informé — source d'erreurs difficiles à diagnostiquer.	Ajout d'une couche de validation explicite (IndexFilterRequest) en amont des filtres, rejetant avec un message clair tout paramètre non reconnu, avant même que la requête n'atteigne la couche de filtrage.
Empêcher qu'une salle (Espace) soit réservée deux fois pour le même créneau, tout en autorisant la mise à jour d'une séance existante sans déclencher un faux conflit avec elle-même.	Combinaison d'une contrainte d'unicité au niveau base de données (espace_id, date, time_range_id) et d'une vérification applicative explicite (Espace::isAvailable()) excluant la séance en cours de modification de la recherche de conflit.
Risque de fuite de données personnelles des stagiaires (CNI, téléphone, adresse, date de naissance) dans les réponses de l'API alors qu'elles ne sont pas systématiquement nécessaires.	Double protection : ces colonnes sont déclarées \$hidden au niveau du modèle Eloquent Stagiaire, en plus d'un contrôle explicite dans StagiaireResource qui ne renvoie que les champs définis dans son contrat de sortie.

Chapitre 8 — Conclusion et perspectives

8.1 Bilan du stage

Ce stage a permis de transformer un ensemble de données académiques dispersées dans des fichiers Excel hétérogènes en un service de données unifié, structuré et interrogeable, répondant directement à la problématique posée en introduction. L'API CMC Data API expose aujourd'hui l'intégralité du domaine académique du centre — pôles, filières, années, groupes, modules, formateurs, stagiaires, affectations, séances et notes — au travers d'un contrat REST clair, versionné, doté d'un système de filtrage, de tri et d'inclusion de relations entièrement contrôlé par liste blanche.

Sur le plan technique, ce projet a constitué une mise en pratique concrète et approfondie des principes d'architecture logicielle en couches (séparation routes / validation / filtrage / service / sérialisation), des bonnes pratiques de sécurité d'une API REST (limitation de débit, protection CSRF/CORS, validation stricte des entrées, journalisation), ainsi que des techniques d'import de données de masse tolérant les erreurs partielles. Le module de tests automatisés développé en parallèle garantit que ces comportements restent corrects à mesure que le projet évoluera.

Sur le plan personnel, ce stage a été l'occasion de manipuler, dans un contexte professionnel réel, l'ensemble de la chaîne de développement d'une API : depuis l'analyse de données métier imparfaites jusqu'à la mise en production d'un service testé, sécurisé et documenté, en passant par la conception d'un modèle de données respectant des contraintes d'intégrité réelles.

8.2 Perspectives d'évolution (phase 2)

Plusieurs axes d'amélioration, déjà anticipés dans la conception actuelle, ont été identifiés pour la suite du projet :

11. Authentification et gestion des rôles : l'architecture de sécurité (clé client, quotas, journal d'audit) a été conçue pour accueillir nativement un utilisateur authentifié sans réécriture — l'implémentation de Laravel Sanctum ou d'un système de jetons API, assortie d'une distinction de rôles (administrateur, coordination, formateur), constitue la prochaine étape naturelle.
12. Webhooks ou notifications en temps réel lors d'événements clés (note saisie, séance créée), pour permettre à des services tiers de réagir sans avoir à interroger l'API en continu (polling).
13. Tableau de bord analytique enrichi, exploitant directement les nouveaux filtres (ex. taux de réussite par filière, taux de réalisation horaire par module) pour fournir des indicateurs de pilotage à la direction du centre.
14. Extension du module d'import à d'autres types de fichiers sources (planification, évaluations détaillées) selon le même schéma de tolérance aux erreurs déjà en place.

15. Ouverture progressive de l'API à d'autres pôles du centre, en généralisant le modèle de données au-delà du seul Pôle Digital et Intelligence Artificielle.

En définitive, ce projet pose les fondations techniques d'un système d'information académique pérenne pour le centre, conçu pour évoluer sans remise en cause de son architecture, et déjà prêt à servir de source de vérité pour d'autres outils internes ou externes.

